

# ■ EXTERNAL AGENT INTEGRATION GUIDE

Step-by-Step Implementation Guide for Instrumenting Custom Agents, Capturing Checkpointed Traces, and Debugging with Black Box

---

This guide walks developers through connecting external AI agents (LangChain, LlamaIndex, CrewAI, AutoGen, or native Python ReAct loops) to the Black Box diagnostic ecosystem. By instrumenting your agent with lightweight observability hooks, Black Box enables automatic root-cause localization, 1-click remediation, and time-travel replay.

## Step 1: Installation & Directory Configuration

Include the `blackbox` package in your project or install your environment:

```
# Set up Black Box trace repository from pathlib import Path from blackbox.capture.store import TraceStore
store = TraceStore(Path('data/traces'))
```

## Step 2: Instrumenting Agent Execution Loops

Wrap your agent's execution cycle with **TraceContext**. Each LLM call, tool invocation, or context retrieval is recorded within a `with ctx.step(...)` block:

```
from blackbox.capture.instrument import TraceContext from blackbox.capture.schema import TaskType, StepType
def run_custom_agent(user_query: str) -> str: with TraceContext(run_id='run-ext-001', task_id='task-42') as ctx: # Step 0: LLM Decision / Plan with ctx.step(StepType.LLM_CALL, inputs={'query': user_query}) as step:
    plan = llm.chat(f'Plan steps for: {user_query}') step.set_output(plan) # Step 1: Tool Call (e.g. database query, calculator, API) with ctx.step(StepType.TOOL_CALL, tool='sql_db', inputs={'sql': 'SELECT ...'}) as step:
    db_rows = database.execute('SELECT ...') step.set_output(db_rows) step.set_tool_call('sql_db', {'sql': 'SELECT ...'}, db_rows) # Step 2: Final Model Synthesis with ctx.step(StepType.LLM_CALL, inputs={'rows': db_rows}) as step:
    final_ans = llm.chat(f'Summarize: {db_rows}') step.set_output(final_ans) # Build immutable trace
    trace = ctx.build_trace( task_type=TaskType.CUSTOM, task_description=user_query, expected_output='Target Answer', final_output=final_ans, success=(final_ans == 'Target Answer') )
    store.save(trace) return final_ans
```

## Step 3: Auto-Capturing Tools with @capture\_tool

For modular tools, simply decorate tool functions with `@capture\_tool`:

```
from blackbox.capture.instrument import capture_tool class AgentToolkit: def __init__(self, trace_ctx):
    self._trace_ctx = trace_ctx @capture_tool('web_search') def search(self, query: str): return
    external_search_api(query)
```

## Step 4: Running Automated AI Fault Diagnosis

When an agent run fails, call the local analyzer to pinpoint the root-cause step in < 1 ms:

```
from blackbox.attribution.model import LocalAttributionModel from blackbox.analysis.local import
analyze_local from blackbox.debug.diagnose import create_diagnosis from blackbox.debug.suggestions import
generate_suggestions from blackbox.analysis.consensus import ConsensusResult model = LocalAttributionModel()
model.load('data/artifacts/model.pkl') # 1. Run local ML diagnosis local_res = analyze_local(trace, model)
consensus = ConsensusResult(local_res.top_step, local_res.scores, True, 'local_only', local_res.confidence)
diagnosis = create_diagnosis(trace, consensus) print(f'Culprit Step: {diagnosis.likely_source}')
print(f'Confidence: {diagnosis.confidence}') print(f'Rationale: {diagnosis.rationale}') # 2. Retrieve
actionable fix suggestions suggestions = generate_suggestions(trace, diagnosis.likely_source) for s in
suggestions: print(f'Fix: {s.title} -> {s.description}')
```

## Step 5: Testing Fixes with Time-Travel Replay

Apply the recommended patch at the suspect step and resume downstream execution without re-running past steps:

```
from blackbox.intervene.patch import StepPatch from blackbox.intervene.branch import create_branch from
blackbox.compare.divergence import find_divergence # Apply patch to culprit step (e.g. override faulty
calculation or SQL) patch = StepPatch( step_index=diagnosis.likely_source, patch_kind='override_output',
new_output=suggestions[0].patch_payload.get('output'), description=suggestions[0].title ) # Resume branch
from checkpoint repaired_trace = create_branch(trace, patch) print(f'Repaired Run Status:
{repaired_trace.success}') print(f'Savings: {repaired_trace.meta.get("replay_savings_pct")}% computation
avoided!') # Compare side-by-side diff = find_divergence(trace, repaired_trace) print(f'Divergence point:
Step {diff.earliest_step_index}')
```

## ■ Best Practices for Production Agent Debugging

- **Always Record Checkpoints:** Attach intermediate memory state so the agent is resumable at any step.
- **Track Input & Output Tokens:** Ensures full visibility into token and cost savings during replay.
- **Use Local ML for Fast Filtering:** Local attribution runs in < 1 ms, filtering 90%+ of traces before invoking expensive LLM judges.
- **Review Trace Diffs:** Always inspect the earliest divergence in the Streamlit UI to verify fix validity.